

AI Customer Support Coaching Assistant

Plain-English project explainer — orchestration, RAG, chunking, embeddings, vector database, and the multi-agent pipeline, mapped to what is actually built in the code.

Prepared as a milestone-review briefing document. Read top to bottom once before the meeting; use the Q&A Cheat Sheet section as your live reference during the review.

1. The 30-Second Pitch

This is a Streamlit web app called **CoachAI**. It sits next to a customer-support agent while they chat with a customer, and in real time it tells the agent: what the customer is feeling, what knowledge-base article to use, how to phrase the reply, and whether the customer is about to escalate or churn. After the conversation ends, it writes a performance report (sentiment journey, resolution quality score, coaching tips) that a manager can review.

It supports three modes: **Simulator Mode** (an AI-generated fake customer chats with you so you can practice), **Manual Mode** (you paste in a real customer's messages), and **Replay Mode** (it replays an old transcript message-by-message so you can see the coaching that would have fired).

2. What Is Orchestration?

General idea: "Orchestration" means having one central controller that decides *which specialized AI agent runs, in what order, with what input, and what to do with its output* — instead of one giant AI model trying to do everything itself. It's the same idea as an orchestra conductor: the conductor doesn't play an instrument, it tells each musician when to come in.

How it works in this codebase:

There are two layers of orchestration here:

- **Orchestrator** (`src/core/orchestrator.py`) — the top-level controller. It owns the session lifecycle: start session → route each customer/agent message → end session → generate the final report. It doesn't do any analysis itself; it just calls the right module at the right time.
- **ConversationManager** (`src/modules/conversation_manager.py`) — the per-turn orchestrator. Every time the customer sends a message, this is the piece that fires off the different agents *in a fixed sequence* and stitches their outputs into one `TurnAnalysis` object that the UI displays.

The per-turn sequence, in order, is:

- 1. **Intent & Sentiment Agent** reads the customer message and classifies intent, sentiment, frustration level, satisfaction trend.
- 2. **Knowledge Recommendation Agent** takes the message, retrieves matching knowledge-base chunks (RAG), and asks the LLM to synthesize an actionable tip from them.
- 3. **Coaching Suggestion Agent** uses the intent analysis to generate a suggested reply / tone feedback for the agent.

- 4. **Escalation Monitor Agent** scores flight/escalation risk using the intent + session history.
- 5. Results are packaged into one `TurnAnalysis` and handed back to the UI, which renders the three-panel console.

*This is a **deterministic, code-defined pipeline** (a fixed call order written in Python), not an autonomous agent that decides its own steps. If asked "is this agentic orchestration or a hard-coded pipeline?" — the honest answer is: **a hard-coded, sequential orchestration pipeline**, where each step calls an LLM to do reasoning, but the **order and wiring** of steps is fixed by the `Orchestrator/ConversationManager` code, not decided at runtime by an agent.*

3. The Multi-Agent Pipeline — Who Does What

The project brief calls for 6 core agents. All 6 are implemented, plus extra bonus agents built on top. Core six:

Agent	File	Job
Customer Simulator	<code>customer_simulator.py</code>	Generates realistic AI customer messages turn-by-turn from a scenario + persona (used in Simulator Mode).
Intent & Sentiment	<code>intent_sentiment.py</code>	Classifies customer intent, sentiment, frustration level (0-1) and satisfaction trend, via LLM call returning JSON.
Knowledge Recommendation	<code>knowledge_recommendation.py</code>	RAG: retrieves top-matching KB chunks for the message, then asks the LLM to synthesize one actionable tip.
Coaching & Response Suggestion	<code>coaching_suggestion.py</code>	Suggests what the agent should say, scores tone/clarity, gives improvement tips.
Escalation Risk Monitor	<code>escalation_monitor.py</code>	Continuously scores escalation likelihood with reasoning + recommended de-escalation strategy.
Post-Interaction Summary	<code>post_interaction_summary.py</code>	After the session ends: writes sentiment journey, resolution quality score, and coaching recommendations.

Bonus / extra agents built beyond the brief (good for demonstrating extra effort): **Coach Calibrator** (learns per-agent when to actually show a coaching tip vs. suppress it, so it doesn't spam a good agent), **Predictive CSAT Agent** (forecasts 1-5 CSAT + churn % live), **Manager Supervisor Agent** (simulates a supervisor who can whisper notes or take over on extreme risk), **Tone Rewriter / De-escalation Agent**, **Auto-KB Agent** (auto-suggests new KB articles from gaps), **Multiverse / Counterfactual Simulator** (replays "what if the agent had said X instead"), plus fraud, compliance, cognitive-load, and viral-threat monitors. These aren't in the official milestone list — mention them as "extra depth added beyond spec" if asked.

4. RAG (Retrieval-Augmented Generation) — the Basics

General idea: An LLM only knows what it was trained on — it has no idea about your company's specific FAQs, policies, or product docs. RAG fixes this without retraining the model: you keep your documents in a searchable store, and for every user question you (1) **retrieve** the most relevant pieces of text, then (2) **feed those pieces into the LLM's prompt** alongside the question, so the model "augments" its answer with real, current, company-specific information instead of guessing.

The two ways to do retrieval (step 1):

- **Semantic / vector retrieval** — turn text into number vectors ("embeddings") that capture meaning, store them in a vector database, and find matches by geometric closeness. Understands synonyms and paraphrasing.
- **Lexical / keyword retrieval** — match on shared words directly (classic search-engine style). Simple, fast, no ML model needed, but misses synonyms ("refund" vs "money back").

***What this project actually uses: lexical / keyword retrieval — not embeddings, not a vector database.** See sections 5 and 6 below for the honest detail on this, because it's the question most likely to trip you up.*

5. Chunking

General idea: you can't hand a whole 50-page PDF to a retrieval system as one block — you split ("chunk") documents into smaller pieces first, so the search can find and return just the relevant paragraph, not the whole document.

What this project does (`src/rag/knowledge_base.py`):

- Ingests `.txt`, `.pdf`, `.docx`, `.json`, `.md`, `.csv`, `.html` files from `data/knowledge_base/`.
- Splits text on blank lines into **paragraphs** first.
- Groups paragraphs together into a chunk until it hits **chunk_size = 512 characters** (configured in `src/core/config.py` — note: this is *characters*, not tokens).
- If a single paragraph is still too long (> 1.5x the chunk size), it falls back to splitting by **sentence** so no chunk is ever excessively long.
- Each chunk gets tokenized (lower-cased, word-split with a regex) into a **set of unique words** — this token set is what gets searched later, instead of a vector embedding.

There is a configured `chunk_overlap = 64` setting in `config.py`, but the current chunker does not actually apply overlap between chunks — worth knowing in case it's probed, and an easy "next improvement" talking point.

6. Am I Using an Embedding Model?

Short honest answer: No — not currently active in the running app.

General idea of an embedding model: it's a small neural network that converts a piece of text into a vector of numbers (e.g. 384 numbers) such that texts with *similar meaning* end up as *nearby vectors*. Common examples: `all-MiniLM-L6-v2` (Sentence-Transformers), OpenAI's `text-embedding-3`, Cohere embeddings.

What's actually in this codebase:

- `src/core/config.py` has a field `embedding_model = "all-MiniLM-L6-v2"` — but grep the codebase and it is **never loaded or called anywhere**. It's a leftover placeholder from an earlier design, not something live.
- The actual retrieval in `knowledge_base.py` uses plain Python: it lower-cases and regex-tokenizes both the query and every chunk, then scores each chunk by **keyword overlap** — `score = (shared words) / (words in query)`, a Jaccard-style overlap score. No neural network, no vector math, no external embedding API call.

- The code comment literally says: *"This entirely replaces ChromaDB and ensures ZERO local ML models are used."* — this was a deliberate lightweight design choice, not an accident.

How to phrase this in the meeting so it sounds like a design decision, not a gap: "For this stage I used a lightweight lexical/keyword retrieval layer instead of a neural embedding model — it needs no extra ML dependency, no GPU, and returns results instantly, which mattered for a real-time coaching tool. The knowledge base is small enough (FAQ-scale) that keyword overlap retrieval works well. The natural next iteration is to swap this for a sentence-embedding model like `all-MiniLM-L6-v2` plus a vector index once the knowledge base grows or we need to handle paraphrased queries."

7. Am I Using a Vector Database?

Short honest answer: No.

General idea: a vector database (Chroma, FAISS, Pinecone, Weaviate, Milvus...) stores the embedding vectors from section 6 and lets you query "find me the N chunks whose vector is closest to this query's vector" efficiently, even across millions of chunks.

What's actually in this codebase:

- All knowledge chunks live in a plain Python list in memory: `self.documents = []` inside the `KnowledgeBase` class — each entry is a dict with the chunk text, its word-token set, and metadata.
- There is a `data/vector_store/` folder and a `vector_store_path` config setting, but it is **empty / unused** — a directory that gets auto-created by `config.py` but nothing ever writes vectors into it.
- Search happens by looping over every chunk in memory and computing the keyword-overlap score — this is fine at FAQ-scale (tens to low hundreds of chunks) but wouldn't scale to a huge document set without moving to a real vector DB.

How to phrase this: "There's no ChromaDB/FAISS/Pinecone in the current build — the knowledge store is in-memory Python, rebuilt each app run from the files in `data/knowledge_base/`. That's sufficient for the current FAQ-scale knowledge set and keeps the app dependency-light; a vector database is the planned upgrade path once we add embeddings."

8. Which LLM Actually Powers the Agents?

Every agent that needs reasoning (intent/sentiment, coaching, knowledge synthesis, CSAT prediction, manager supervisor, etc.) calls one shared helper: `llm_chat()` in `src/core/llm.py`. It talks to the **Groq API** (fast LLM inference hosting), not OpenAI, even though a legacy `openai_api_key` field still exists unused in config.

Model fallback chain (tries each in order until one responds):

- 1. `llama-3.3-70b-versatile` — primary, most capable.
- 2. `llama-3.1-8b-instant` — faster fallback if the 70B model is rate-limited.
- 3. `gemma2-9b-it` — second fallback.
- 4. `llama3-8b-8192` — final fallback.

If no `GROQ_API_KEY` is set at all, `llm_chat()` returns an empty string and each agent has its own rule-based / heuristic fallback so the app degrades gracefully instead of crashing.

9. End-to-End Flow, One Turn

- Customer sends a message (typed, simulated, or replayed).
- `Orchestrator.process_customer_input()` hands it to `ConversationManager`.
- Intent & Sentiment Agent → LLM call → returns intent, sentiment, frustration, trend (JSON).
- Knowledge Recommendation Agent → keyword search over in-memory chunks (RAG retrieval) → LLM call to synthesize a tip (RAG generation).
- Coaching Suggestion Agent → LLM call → suggested reply + tone feedback.
- Escalation Monitor Agent → rule-based scoring using frustration + history → risk level + strategy.
- All four outputs bundle into one `TurnAnalysis` → UI renders it across the 3-panel console.
- Agent sends their real reply → Coaching agent re-scores the actual response quality.
- At session end → Post-Interaction Summary Agent builds the sentiment-journey report + resolution score + coaching recap.

10. Mapping This to the Official Milestone Document

Your brief ("AI-Powered Real-Time Customer Support Coaching Assistant", Vidzai Digital) lists 6 required agents and 7 modules. Status against the actual code:

Required Item	Status
Customer Simulator Agent	Done — <code>customer_simulator.py</code>
Intent & Sentiment Analysis Agent	Done — <code>intent_sentiment.py</code>
Knowledge Recommendation Agent (RAG)	Done, using keyword-retrieval RAG (not vector/embedding-based yet) — <code>knowledge_recommendation.py</code> + <code>rag/knowledge_base.py</code>
Coaching & Response Suggestion Agent	Done — <code>coaching_suggestion.py</code>
Escalation Risk Monitor Agent	Done — <code>escalation_monitor.py</code>

Post-Interaction Summary Agent	Done — post_interaction_summary.py
Session Config & Mode Selection module	Done — session_config.py (Simulator/Manual/Replay)
KB Ingestion & RAG Pipeline module	Done, lexical retrieval only (no embeddings/vector DB) — rag/ingest.py, rag/knowledge_base.py
Multi-Agent Coaching Pipeline / Orchestration	Done — orchestrator.py + conversation_manager.py
Live 3-Panel Console UI	Done — ui/app.py, ui/panels.py
Escalation Alert Module	Done — escalation_monitor.py + UI alert overlay
Post-Interaction Report & Analytics module	Done — performance_analytics.py

Bottom line: **every required agent and module exists and works**. The one gap versus the literal brief wording ("implement chunking, embedding, and vector indexing") is that embedding + vector indexing were substituted with a lighter keyword-based retrieval approach that still fulfills RAG's purpose (grounding answers in real KB content). Flag that as a conscious engineering trade-off with a clear upgrade path, not an oversight.

11. Live Q&A Cheat Sheet

Read this section right before you walk in. Answers are written so you can say them almost verbatim.

Q: What is orchestration in your project?

Orchestration is the layer that decides which AI agent runs, in what order, and how their outputs combine. In my project it's two pieces: the Orchestrator manages the session lifecycle (start, process each turn, end, generate report), and the ConversationManager runs the fixed per-turn sequence — Intent Agent, then Knowledge Agent, then Coaching Agent, then Escalation Agent — and merges all four results into one TurnAnalysis object that the UI displays.

Q: Is your pipeline agentic or hard-coded?

It's a hard-coded, deterministic pipeline — the call order is fixed in Python code. Each individual agent uses an LLM to reason about its specific task, but the wiring between agents isn't decided dynamically by an AI at runtime.

Q: Are you using an embedding model?

Not in the active retrieval path. I use lightweight keyword/lexical overlap matching for retrieval instead of neural embeddings — it's dependency-light and instant, which fits a real-time coaching use case at FAQ-scale. Swapping in a sentence-embedding model like all-MiniLM-L6-v2 is the planned next step once the knowledge base grows.

Q: What is chunking and how did you do it?

Chunking is splitting long documents into smaller pieces so retrieval can return just the relevant part. I split ingested files on blank lines into paragraphs, then group paragraphs up to a 512-character chunk size, falling back to sentence-level splitting if a single paragraph is still too long.

Q: What vector database are you using?

None currently — chunks are held in an in-memory Python list with their tokenized words, searched by keyword overlap. There's a reserved vector_store folder for a future Chroma/FAISS upgrade, but it isn't populated yet.

Q: How does your RAG actually work end-to-end?

Retrieval: the customer's message is tokenized and compared against every knowledge chunk's token set, scored by overlap, top matches returned. Generation: those matched chunks are inserted into a prompt sent to the LLM, which synthesizes one short, actionable recommendation for the agent — so the answer is grounded in real KB content instead of the model guessing.

Q: Which LLM are you using?

Groq-hosted Llama 3.3 70B as the primary model, with automatic fallback to Llama 3.1 8B, Gemma2 9B, and Llama3 8B if the primary is rate-limited, so the app doesn't fail if one model is busy.

Q: How many agents do you have and what do they do?

Six core agents as per the brief — Customer Simulator, Intent & Sentiment, Knowledge Recommendation, Coaching Suggestion, Escalation Monitor, and Post-Interaction Summary — plus extra agents I added beyond spec like a Coach Calibrator that learns when to actually show a tip, a Predictive CSAT/churn agent, and a Manager Supervisor agent that simulates live escalation takeover.

Q: What happens if the knowledge base has no good match?

The Knowledge Agent checks the top relevance score; if it's below 0.45 it raises a 'Knowledge Base Gap Detected' flag so the team knows to add an article for that topic, instead of silently returning a weak match.

Q: What's the single biggest technical trade-off you made?

Choosing keyword-based retrieval over embeddings + a vector database. It made the app fast and dependency-free to build and demo, at the cost of missing synonym/paraphrase matches that a semantic embedding search would catch. It's a clean, well-scoped upgrade for the next milestone.

Generated from the actual codebase at E:\Projects\customer-support-coach on 2026-07-28 — file references (src/core/orchestrator.py, src/rag/knowledge_base.py, src/core/llm.py, src/core/config.py, src/agents/*.py) are accurate as of this date.